Bölüm 3: Nash Dengesi

Algoritmik Oyun Kuramı

Biçimsel tanım · En iyi tepki · Baskın strateji eleme · Karışık strateji · Varlık teoremi · Cournot · Denge seçimi · Korelasyonlu denge · Minimaks · PPAD

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

TEMELLER

Öğrenme Çıktıları

Tanım

Nash dengesini biçimsel olarak tanımlamak ve en iyi tepki (best response) kavramıyla ilişkilendirmek.

Saf denge bulma

Kazanç matrisinde en iyi tepkileri işaretlemek ve yinelemeli baskın strateji eleme uygulamak.

Karışık denge

Kayıtsızlık ilkesiyle karışık strateji dengesini hesaplamak; sıfır toplamlı oyunda minimaks değerini bulmak.

Varlık ve karmaşıklık

Nash'in varlık teoremini, çoklu denge sorununu ve denge hesaplamanın PPAD-tamlığını yorumlamak.

TEMELLER

John Nash Kimdir?

- Amerikalı matematikçi **John Forbes Nash Jr.** (1928–2015).
- 1950'de Princeton'da 28 sayfalık doktora tezi *Non-Cooperative Games*; aynı yıl *PNAS*'ta bir sayfalık varlık ispatı.
- 1994 Nobel Ekonomi Ödülü (Harsanyi ve Selten ile birlikte).
- 2015 Abel Ödülü: kısmi diferansiyel denklemlere katkıları.
- Hayatı *A Beautiful Mind* (2001) filmine konu oldu.



John F. Nash Jr.

Katkısı: von Neumann'ın **iki kişilik sıfır toplamlı** oyunlar için kurduğu çözümü, **n kişilik genel** (sıfır toplamlı olmayan) oyunlara taşıdı.

TEMELLER

Stratejik Davranış ve Uygulama Alanları

- Oyuncular kendi kazançlarını en büyüklemeye çalışır.
- Herkesin kazancı **diğerlerinin seçimlerine de** bağlıdır: “Ben ne yapmalıyım?” sorusu “Onlar ne yapacak?” sorusundan ayrılamaz.

Fiyat rekabeti
İki firma fiyat indirir; ikisi de kâr kaybeder.
`Mahkûmun İkilemi yapısı`

Reklam / silahlanma
Herkes harcar, göreli konum değişmez.
`baskın strateji: harca`

Penaltı atışı
Atıcı ve kaleci tahmin edilemez olmalı.
`karışık strateji, sıfır toplamlı`

Trafikte şerit seçimi
Herkes hızlı şeride geçerse o şerit yavaşlar.
`tıkanıklık oyunu`

Standart seçimi
Aynı teknolojiyi seçmek herkese kazandırır.
`koordinasyon, çoklu denge`

Kaynak paylaşımı
Ortak bant genişliği, sunucu, yol.
`Anarşinin Bedeli`

TEMELLER

Nash Dengesi: Biçimsel Tanım

Normal form oyun $G = (N, (S_i), (u_i))$:

- $N = \{1, \ldots, n\}$ oyuncular; S_i oyuncu i'nin strateji kümesi.
- Strateji profili $s = (s_1, \ldots, s_n)$; s_{-i}: i dışındakilerin stratejileri.
- $u_i(s)$: i'nin kazancı (u_i: kazanç fonksiyonu).

Nash dengesi: s^* profili, her oyuncu i ve her $s_i \in S_i$ için

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*)$$

Tek taraflı sapma

Diğerleri sabitken hiçbir oyuncu stratejisini değiştirerek kazancını artıramaz.

Kendini doğrulayan anlaşma

Herkes diğerlerinin uyacağına inanıyorsa, kendisi de uyar.

Kararlılık ≠ verimlilik

Denge adalet ya da verimlilik (Pareto optimallik) garantisi vermez.

EN İYİ TEPKİ

En İyi Tepki (Best Response)

- Oyuncu i'nin, diğerlerinin s_{-i} seçimine **en iyi tepkisi**: kazancını en yüksek yapan stratejiler kümesi

$$BR_i(s_{-i}) = \text{argmax}_{s_i \in S_i} u_i(s_i, s_{-i})$$

- Eşdeğer tanım: s* Nash dengesidir ⇔ her i için $s_i^* \in BR_i(s_{-i}^*)$. Denge = **karşılıklı en iyi tepki**.

1. Satır oyuncusu
Her *sütunda*, satır oyuncusunun en büyük kazancını (u_1) işaretle.

2. Sütun oyuncusu
Her *satırda*, sütun oyuncusunun en büyük kazancını (u_2) işaretle.

3. Kesişim
İki kazancı da işaretli hücreler **saf strateji Nash dengeleridir.**

m × n matriste maliyet O(mn): her hücre bir sütun maksimumu ve bir satır maksimumuyla karşılaştırılır.

Basit Örnek: En İyi Tepkileri İşaretle

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Hücrede (u_1, u_2): önce satır, sonra sütun oyuncusunun kazancı. Henüz işaret yok.

Basit Örnek: En İyi Tepkileri İşaretle

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(**4** , 3)	(0 , 2)	(1 , 1)
B	(2 , 1)	(3 , 3)	(0 , 2)
C	(1 , 4)	(2 , 0)	(3 , 2)

Sütun oyuncusu **X** oynarsa satır oyuncusu u_1 değerlerini karşılaştırır: A:4, B:2, C:1 → en iyisi **A** (4).

Basit Örnek: En İyi Tepkileri İşaretle

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(**4**, 3)	(0, 2)	(1, 1)
B	(2, 1)	(**3**, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Sütun oyuncusu **Y** oynarsa satır oyuncusu u_1 değerlerini karşılaştırır: A:0, B:3, C:2 → en iyisi **B** (3).

Basit Örnek: En İyi Tepkileri İşaretle

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(**4**, 3)	(0, 2)	(1, 1)
B	(2, 1)	(**3**, 3)	(0, 2)
C	(1, 4)	(2, 0)	(**3**, 2)

Sütun oyuncusu **Z** oynarsa satır oyuncusu u_1 değerlerini karşılaştırır: A:1, B:0, C:3 → en iyisi **C** (3).

Basit Örnek: En İyi Tepkileri İşaretle

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(**4**, **3**)	(0, 2)	(1, 1)
B	(2, 1)	(**3**, 3)	(0, 2)
C	(1, 4)	(2, 0)	(**3**, 2)

Satır oyuncusu **A** oynarsa sütun oyuncusu u_2 değerlerini karşılaştırır: X:3, Y:2, Z:1 → en iyisi **X** (3).

Basit Örnek: En İyi Tepkileri İşaretle

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	**(4, 3)**	(0, 2)	(1, 1)
B	(2, 1)	**(3, 3)**	(0, 2)
C	(1, 4)	(2, 0)	**(3**, 2)

Satır oyuncusu **B** oynarsa sütun oyuncusu u_2 değerlerini karşılaştırır: X:1, Y:3, Z:2 → en iyisi **Y** (3).

Basit Örnek: En İyi Tepkileri İşaretle

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(**4** , **3**)	(0 , 2)	(1 , 1)
B	(2 , 1)	(**3** , **3**)	(0 , 2)
C	(1 , **4**)	(2 , 0)	(**3** , 2)

Satır oyuncusu **C** oynarsa sütun oyuncusu u_2 değerlerini karşılaştırır: X:4, Y:0, Z:2 → en iyisi **X** (4).

Basit Örnek: En İyi Tepkileri İşaretle

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(**4**, **3**)	(0, 2)	(1, 1)
B	(2, 1)	(**3**, **3**)	(0, 2)
C	(1, **4**)	(2, 0)	(**3**, 2)

İki kazancı da işaretli hücreler: **(A, X) = (4, 3)** ve **(B, Y) = (3, 3)** → iki saf Nash dengesi. (C, Z) gibi hücrelerde en az bir oyuncu sapmak ister.

Basit Örnek: En İyi Tepkileri İşaretle

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(**4**, **3**)	(0, 2)	(1, 1)
B	(2, 1)	(**3**, **3**)	(0, 2)
C	(1, **4**)	(2, 0)	(**3**, 2)

Sapma testi (A, X): satır oyuncusu B veya C'ye geçerse 2 ya da 1 alır (< 4); sütun oyuncusu Y veya Z'ye geçerse 2 ya da 1 alır (< 3). Kimse kazanamaz ✓

EN İYİ TEPKİ

Hatırlatma: Üç Klasik 2×2 Oyun

Mahkûmun İkilemi

Satır \ Sütun	S	İ
S	−1 , −1	−3 , **0**
İ	**0** , −3	**−2** , **−2**

S: Sessiz (işbirliği), İ: İhanet. Kazanç = −(hapis yılı).
Tek denge **(İ, İ)**; (S, S) ikisine de daha iyi → denge Pareto optimal değil.

Cinsiyetler Savaşı

Satır \ Sütun	M	K
M	**3** , **2**	0 , 0
K	0 , 0	**2** , **3**

İki saf denge; ayrıca karışık denge: satır M'yi 3/5, sütun M'yi 2/5 olasılıkla seçer (%52 buluşamama).

Yazı-Tura Eşleştirme

Satır \ Sütun	Y	T
Y	**1** , −1	−1 , **1**
T	−1 , **1**	**1** , −1

İşaretler hiçbir hücrede çakışmaz: **saf denge yok**. Tek denge karışık: iki oyuncu da ½–½.

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

A: satır oyuncusunun, B: sütun oyuncusunun kazanç matrisi. m = n = 3, **denge = []**.

Adım 1 / 13

```
1 def saf_nash(A, B):
▶       m, n, denge = len(A), len(A[0]), []
3       for i in range(m):
4           for j in range(n):
5               br1 = max(A[k][j] for k in range(m))
6               br2 = max(B[i][l] for l in range(n))
7               if A[i][j] == br1 and B[i][j] == br2:
8                   denge.append((i, j))
9       return denge
```

DEĞİŞKENLER

`m = 3` `n = 3` `denge = []`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Hücre **(A, X)**: A = 4 = sütun maks. ve B = 3 = satır maks. → koşul **doğru**.

Adım 2 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
▶             br1 = max(A[k][j] for k in range(m))
▶             br2 = max(B[i][l] for l in range(n))
▶             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 0` `j = 0` `A[i][j] = 4` `br1 = 4` `B[i][j] = 3` `br2 = 3` `denge = []`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

(0, 0) listeye eklenir: **denge = [(0, 0)]**.

Adım 3 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
5             br1 = max(A[k][j] for k in range(m))
6             br2 = max(B[i][l] for l in range(n))
7             if A[i][j] == br1 and B[i][j] == br2:
▶                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 0` `j = 0` `denge = [(0, 0)]`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Hücre **(A, Y)**: A = 0 < 3 ve B = 2 < 3 → koşul yanlış, denge değil.

Adım 4 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
▶             br1 = max(A[k][j] for k in range(m))
▶             br2 = max(B[i][l] for l in range(n))
▶             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 0` `j = 1` `A[i][j] = 0` `br1 = 3` `B[i][j] = 2` `br2 = 3` `denge = [(0, 0)]`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Hücre **(A, Z)**: A = 1 < 3 ve B = 1 < 3 → koşul yanlış, denge değil.

Adım 5 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
▶             br1 = max(A[k][j] for k in range(m))
▶             br2 = max(B[i][l] for l in range(n))
▶             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 0` `j = 2` `A[i][j] = 1` `br1 = 3` `B[i][j] = 1` `br2 = 3` `denge = [(0, 0)]`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Hücre **(B, X)**: A = 2 < 4 ve B = 1 < 3 → koşul yanlış, denge değil.

Adım 6 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
▶             br1 = max(A[k][j] for k in range(m))
▶             br2 = max(B[i][l] for l in range(n))
▶             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 1` `j = 0` `A[i][j] = 2` `br1 = 4` `B[i][j] = 1` `br2 = 3` `denge = [(0, 0)]`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Hücre **(B, Y)**: A = 3 = sütun maks. ve B = 3 = satır maks. → koşul **doğru**.

Adım 7 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
▶             br1 = max(A[k][j] for k in range(m))
▶             br2 = max(B[i][l] for l in range(n))
▶             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 1` `j = 1` `A[i][j] = 3` `br1 = 3` `B[i][j] = 3` `br2 = 3` `denge = [(0, 0)]`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

(1, 1) listeye eklenir: **denge = [(0, 0), (1, 1)]**.

Adım 8 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
5             br1 = max(A[k][j] for k in range(m))
6             br2 = max(B[i][l] for l in range(n))
7             if A[i][j] == br1 and B[i][j] == br2:
▶                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

i = 1 j = 1

denge = [(0, 0), (1, 1)]

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Hücre **(B, Z)**: A = 0 < 3 ve B = 2 < 3 → koşul yanlış, denge değil.

Adım 9 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
▶             br1 = max(A[k][j] for k in range(m))
▶             br2 = max(B[i][l] for l in range(n))
▶             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 1` `j = 2` `A[i][j] = 0` `br1 = 3` `B[i][j] = 2` `br2 = 3` `denge = [(0, 0), (1, 1)]`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Hücre **(C, X)**: A = 1 < 4 → koşul yanlış, denge değil.

Adım 10 / 13

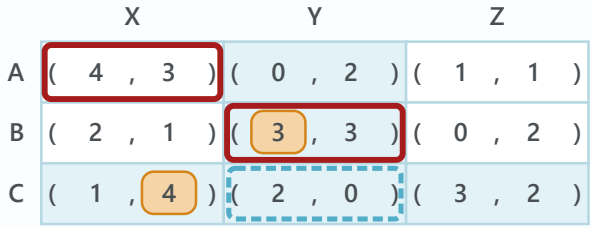
```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
▶             br1 = max(A[k][j] for k in range(m))
▶             br2 = max(B[i][l] for l in range(n))
▶             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 2` `j = 0` `A[i][j] = 1` `br1 = 4` `B[i][j] = 4` `br2 = 4` `denge = [(0, 0), (1, 1)]`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul



Hücre **(C, Y)**: A = 2 < 3 ve B = 0 < 4 → koşul yanlış, denge değil.

Adım 11 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
▶             br1 = max(A[k][j] for k in range(m))
▶             br2 = max(B[i][l] for l in range(n))
▶             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 2` `j = 1` `A[i][j] = 2` `br1 = 3` `B[i][j] = 0` `br2 = 4`

`denge = [(0, 0), (1, 1)]`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Hücre **(C, Z)**: B = 2 < 4 → koşul yanlış, denge değil.

Adım 12 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
▶             br1 = max(A[k][j] for k in range(m))
▶             br2 = max(B[i][l] for l in range(n))
▶             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
9     return denge
```

DEĞİŞKENLER

`i = 2` `j = 2` `A[i][j] = 3`

`br1 = 3` `B[i][j] = 2` `br2 = 4`

`denge = [(0, 0), (1, 1)]`

EN İYİ TEPKİ

Kod İzleme: Saf Nash Dengelerini Bul

	X	Y	Z
A	(4, 3)	(0, 2)	(1, 1)
B	(2, 1)	(3, 3)	(0, 2)
C	(1, 4)	(2, 0)	(3, 2)

Tüm 9 hücre tarandı: **return [(0, 0), (1, 1)]** → (A, X) ve (B, Y). Toplam iş O(mn·(m+n)); maksimumlar önceden hesaplanırsa O(mn).

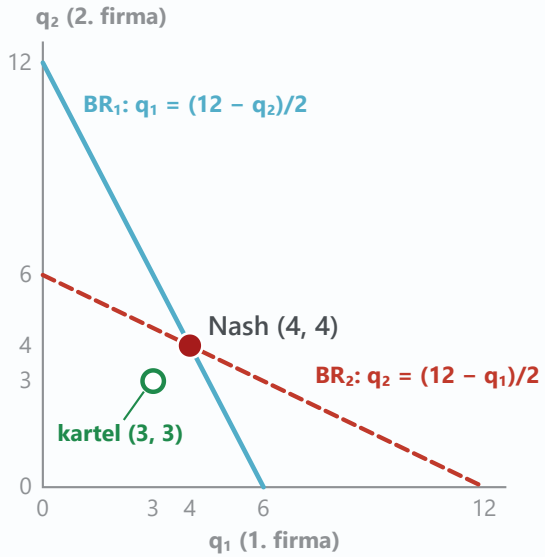
Adım 13 / 13

```
1 def saf_nash(A, B):
2     m, n, denge = len(A), len(A[0]), []
3     for i in range(m):
4         for j in range(n):
5             br1 = max(A[k][j] for k in range(m))
6             br2 = max(B[i][l] for l in range(n))
7             if A[i][j] == br1 and B[i][j] == br2:
8                 denge.append((i, j))
▶     return denge
```

DEĞİŞKENLER

`denge = [(0, 0), (1, 1)]`

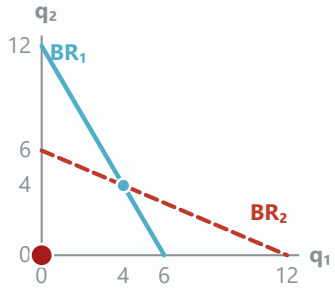
Sürekli Stratejiler: Cournot Duopolü



- İki firma aynı anda üretim miktarı seçer: $q_1, q_2 \geq 0$ (strateji kümesi sonsuz). Fiyat $P = 12 - (q_1 + q_2)$, maliyet 0.
- Kâr $u_1 = q_1 (12 - q_1 - q_2)$. Türev: $12 - 2q_1 - q_2 = 0 \Rightarrow$ **$BR_1(q_2) = (12 - q_2)/2$**.
- Nash dengesi = en iyi tepki doğrularının kesişimi: **$q_1 = q_2 = 4$**, fiyat 4, kâr 16'şar.
- Kartel (3'er üretim) ikisine de 18 verir ama dengede değildir: $BR_1(3) = 4{,}5$ ve sapan firma 20,25 alır.

Sonlu matris yerine sürekli strateji: en iyi tepki bir **fonksiyon**, denge bir **sabit nokta** ($q^* = BR(BR(q^*))$). Cournot, 1838.

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta
q_1	0
q_2	0

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

Başlangıç profili $(q_1, q_2) = (0, 0)$. Firmalar sırayla karşı tarafın son miktarına en iyi tepki verecek.

Adım 1 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
2     return (a - q_diger) / 2
3
▶ def dinamik(q1, q2, tur):
5     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

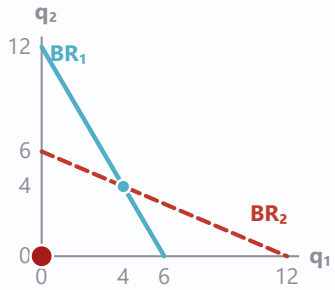
DEĞİŞKENLER

`q1 = 0` `q2 = 0` `tur = 3`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta
q_1	0
q_2	0

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

Döngü: t = 0. tur başlıyor.

Adım 2 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
2     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
▶       for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

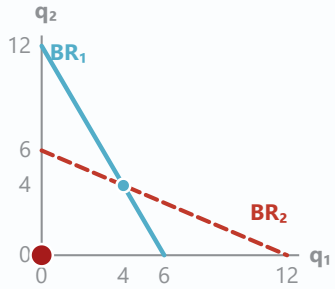
DEĞİŞKENLER

`tur = 3` `t = 0` `q1 = 0` `q2 = 0`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta
q_1	0
q_2	0

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

br(0): (12 − 0)/2 = **6**. 1. firma, rakibin son miktarına göre kârını en büyükler.

Adım 3 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
▶     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

DEĞİŞKENLER

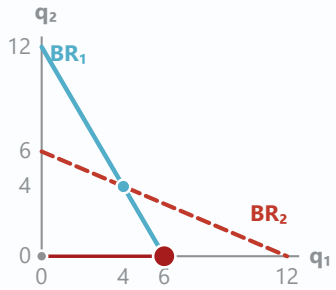
`q_diger = 0` `a = 12`

ÇAĞRI YIĞINI

`br(0)`

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1
q_1	0	6
q_2	0	0

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

q1 = 6.0 atanır; profil (6, 0). Yol, BR_1 doğrusuna yatay gider.

Adım 4 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
2     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
▶         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

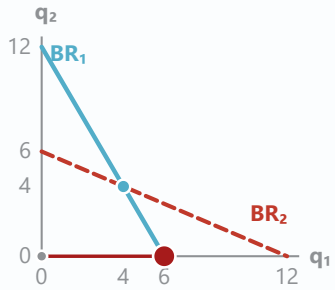
DEĞİŞKENLER

`tur = 3` `t = 0` `q1 = 6.0` `q2 = 0`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1
q_1	0	6
q_2	0	0

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

br(6): (12 − 6)/2 = **3**. 2. firma, rakibin son miktarına göre kârını en büyükler.


```
1 def br(q_diger, a=12):      # en iyi tepki
▶     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

DEĞİŞKENLER

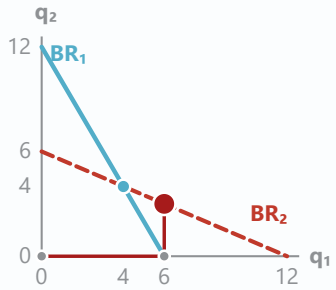
`q_diger = 6.0` `a = 12`

ÇAĞRI YIĞINI

`br(6.0)`

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2
q_1	0	6	6
q_2	0	0	3

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

q2 = 3.0 atanır; profil (6, 3). Yol, BR_2 doğrusuna dikey gider.


```
1 def br(q_diger, a=12):      # en iyi tepki
2     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
▶         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

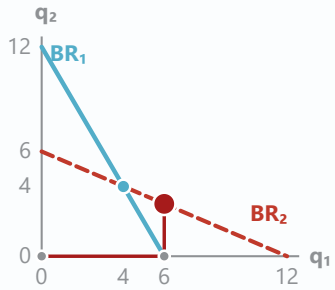
DEĞİŞKENLER

`tur = 3` `t = 0` `q1 = 6.0` `q2 = 3.0`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2
q_1	0	6	6
q_2	0	0	3

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

Döngü: t = 1. tur başlıyor.

Adım 7 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
2     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
▶     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

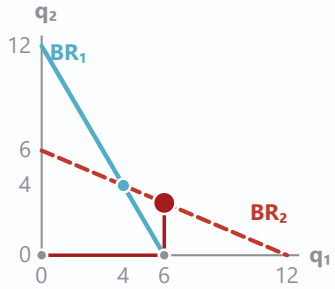
DEĞİŞKENLER

`tur = 3` `t = 1` `q1 = 6.0` `q2 = 3.0`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2
q_1	0	6	6
q_2	0	0	3

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

br(3): (12 − 3)/2 = **4,5**. 1. firma, rakibin son miktarına göre kârını en büyükler.

Adım 8 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
▶     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

DEĞİŞKENLER

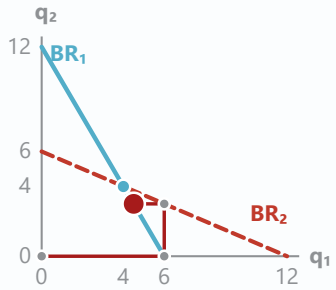
`q_diger = 3.0` `a = 12`

ÇAĞRI YIĞINI

`br(3.0)`

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2	t1·F1
q_1	0	6	6	4,5
q_2	0	0	3	3

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

q1 = 4.5 atanır; profil (4,5, 3). Yol, BR_1 doğrusuna yatay gider.


```
1 def br(q_diger, a=12):      # en iyi tepki
2     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
▶         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

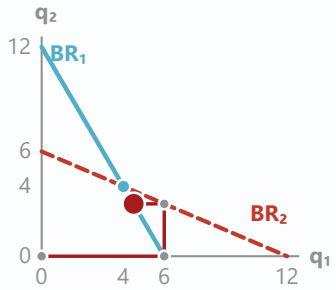
DEĞİŞKENLER

`tur = 3` `t = 1` `q1 = 4.5` `q2 = 3.0`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2	t1·F1
q_1	0	6	6	4,5
q_2	0	0	3	3

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

br(4,5): (12 − 4,5)/2 = **3,75**. 2. firma, rakibin son miktarına göre kârını en büyükler.

Adım 10 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
▶     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

DEĞİŞKENLER

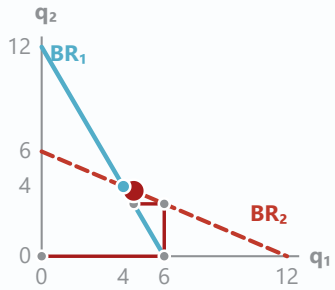
`q_diger = 4.5` `a = 12`

ÇAĞRI YIĞINI

`br(4.5)`

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2	t1·F1	t1·F2
q_1	0	6	6	4,5	4,5
q_2	0	0	3	3	3,75

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

q2 = 3.75 atanır; profil (4,5, 3,75). Yol, BR_2 doğrusuna dikey gider.

Adım 11 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
2     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
▶         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

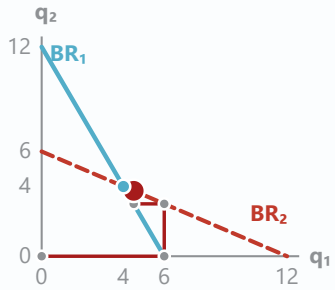
DEĞİŞKENLER

`tur = 3` `t = 1` `q1 = 4.5` `q2 = 3.75`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2	t1·F1	t1·F2
q_1	0	6	6	4,5	4,5
q_2	0	0	3	3	3,75

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

Döngü: t = 2. tur başlıyor.

Adım 12 / 17

```
1  def br(q_diger, a=12):      # en iyi tepki
2      return (a - q_diger) / 2
3
4  def dinamik(q1, q2, tur):
▶      for t in range(tur):
6          q1 = br(q2)         # 1. firma uyum sağlar
7          q2 = br(q1)         # 2. firma uyum sağlar
8      return q1, q2
```

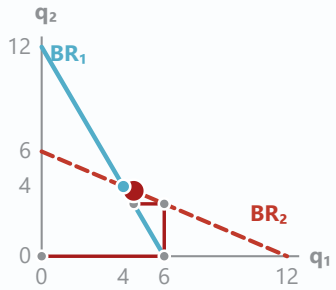
DEĞİŞKENLER

`tur = 3` `t = 2` `q1 = 4.5` `q2 = 3.75`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2	t1·F1	t1·F2
q_1	0	6	6	4,5	4,5
q_2	0	0	3	3	3,75

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

br(3,75): (12 − 3,75)/2 = **4,125**. 1. firma, rakibin son miktarına göre kârını en büyükler.

Adım 13 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
▶     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

DEĞİŞKENLER

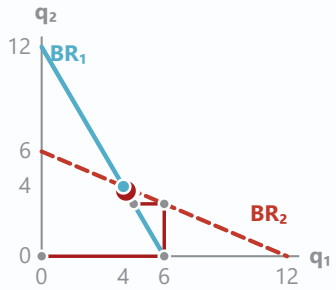
`q_diger = 3.75` `a = 12`

ÇAĞRI YIĞINI

`br(3.75)`

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2	t1·F1	t1·F2	t2·F1
q_1	0	6	6	4,5	4,5	4,125
q_2	0	0	3	3	3,75	3,75

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

q1 = 4.125 atanır; profil (4,125, 3,75). Yol, BR_1 doğrusuna yatay gider.

Adım 14 / 17

```
1  def br(q_diger, a=12):      # en iyi tepki
2      return (a - q_diger) / 2
3
4  def dinamik(q1, q2, tur):
5      for t in range(tur):
▶          q1 = br(q2)         # 1. firma uyum sağlar
7          q2 = br(q1)         # 2. firma uyum sağlar
8      return q1, q2
```

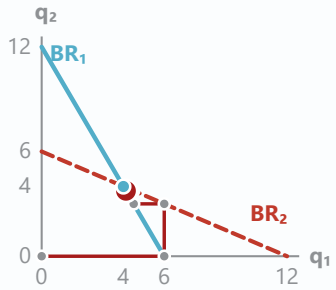
DEĞİŞKENLER

`tur = 3` `t = 2` `q1 = 4.125` `q2 = 3.75`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2	t1·F1	t1·F2	t2·F1
q_1	0	6	6	4,5	4,5	4,125
q_2	0	0	3	3	3,75	3,75

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

br(4,125): (12 − 4,125)/2 = **3,9375**. 2. firma, rakibin son miktarına göre kârını en büyükler.

Adım 15 / 17

```
1 def br(q_diger, a=12):      # en iyi tepki
▶     return (a - q_diger) / 2
3
4 def dinamik(q1, q2, tur):
5     for t in range(tur):
6         q1 = br(q2)         # 1. firma uyum sağlar
7         q2 = br(q1)         # 2. firma uyum sağlar
8     return q1, q2
```

DEĞİŞKENLER

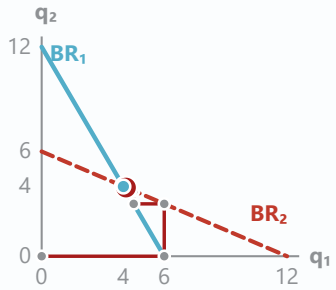
`q_diger = 4.125` `a = 12`

ÇAĞRI YIĞINI

`br(4.125)`

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2	t1·F1	t1·F2	t2·F1	t2·F2
q_1	0	6	6	4,5	4,5	4,125	4,125
q_2	0	0	3	3	3,75	3,75	3,9375

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

q2 = 3.9375 atanır; profil (4,125, 3,9375). Yol, BR_2 doğrusuna dikey gider.

Adım 16 / 17

```
1  def br(q_diger, a=12):      # en iyi tepki
2      return (a - q_diger) / 2
3
4  def dinamik(q1, q2, tur):
5      for t in range(tur):
6          q1 = br(q2)         # 1. firma uyum sağlar
▶          q2 = br(q1)         # 2. firma uyum sağlar
8      return q1, q2
```

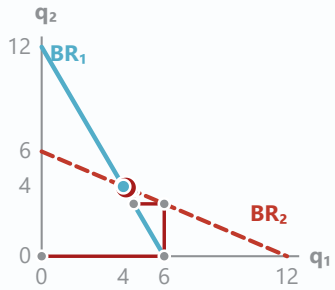
DEĞİŞKENLER

`tur = 3` `t = 2` `q1 = 4.125` `q2 = 3.9375`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

Kod İzleme: En İyi Tepki Dinamiği (Cournot)



Profil geçmişi (F1 / F2: hangi firma uyum sağladı)

	başta	t0·F1	t0·F2	t1·F1	t1·F2	t2·F1	t2·F2
q_1	0	6	6	4,5	4,5	4,125	4,125
q_2	0	0	3	3	3,75	3,75	3,9375

Nash dengesi (4, 4); nokta her adımda ona yaklaşır.

3 turdan sonra (4,125; 3,9375). Uzaklık her turda 4'te 1'ine iner (2 → 0,5 → 0,125): dinamik Nash dengesine (4, 4) **yakınsar**.

Adım 17 / 17

```
1  def br(q_diger, a=12):      # en iyi tepki
2      return (a - q_diger) / 2
3
4  def dinamik(q1, q2, tur):
5      for t in range(tur):
6          q1 = br(q2)         # 1. firma uyum sağlar
7          q2 = br(q1)         # 2. firma uyum sağlar
▶      return q1, q2
```

DEĞİŞKENLER

`tur = 3` `t = 2` `q1 = 4.125` `q2 = 3.9375`

`dönüş = (4.125, 3.9375)`

ÇAĞRI YIĞINI

`dinamik(0, 0, 3)`

BASKIN STRATEJİ ELEME

Baskın ve Baskılanan Strateji

Kesin baskılama (strict dominance)

s_i', s_i'yi kesin baskılar: her s_{-i} için $u_i(s_i', s_{-i}) > u_i(s_i, s_{-i})$.

`Rasyonel oyuncu kesin baskılanan stratejiyi asla oynamaz.`

Zayıf baskılama (weak dominance)

Her s_{-i} için $\geq$ ve en az bir s_{-i} için $>$.

`Silinirse bazı dengeler kaybolabilir; sonuç eleme sırasına bağlı olabilir.`

- **Baskın strateji**: diğer tüm stratejilerini baskılayan strateji (Mahkûmun İkilemi'nde İhanet).
- **Yinelemeli eleme (IESDS)**: kesin baskılanan stratejileri sil, küçülen oyunda tekrarla. Herkes rasyonel *ve* bunu herkes biliyor (ortak bilgi) varsayımı.
- Kesin baskılamada **sıra önemsizdir**; hiçbir Nash dengesi silinmez. Tek hücre kalırsa o hücre oyunun **tek** Nash dengesidir.
- 2/3 ortalama oyunu: 100 → 66,7 → 44,4 → … → 0.

Yinelemeli Eleme: 3×3 Örnek

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(3, 2)	(3, 3)	(1, 4)
B	(3, 2)	(0, 4)	(5, 3)
C	(2, 1)	(2, 0)	(6, 6)

Başlangıç: hiçbir satır diğerini kesin baskılamıyor (ör. A ile B: 3 = 3). Önce sütun oyuncusuna bakalım.

Yinelemeli Eleme: 3×3 Örnek

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(3, 2)	(3, 3)	(1, 4)
B	(3, 2)	(0, 4)	(5, 3)
C	(2, 1)	(2, 0)	(6, 6)

Kalan satırlarda **Z**, **X**'i kesin baskılar (u_2: 4 > 2, 3 > 2, 6 > 1) → X elenir.

Yinelemeli Eleme: 3×3 Örnek

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(3 , 2)	(3 , 3)	(1 , 4)
B	(3 , 2)	(0 , 4)	(5 , 3)
C	(2 , 1)	(2 , 0)	(6 , 6)

Kalan sütunlarda **C**, **B**'yi kesin baskılar (u_1: 2 > 0, 6 > 5) → B elenir.

Yinelemeli Eleme: 3×3 Örnek

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(3, 2)	(3, 3)	(1, 4)
B	(3, 2)	(0, 4)	(5, 3)
C	(2, 1)	(2, 0)	(6, 6)

Kalan satırlarda **Z**, **Y**'yi kesin baskılar (u_2: 4 > 3, 6 > 0) → Y elenir.

Yinelemeli Eleme: 3×3 Örnek

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(3, 2)	(3, 3)	(1, 4)
B	(3, 2)	(0, 4)	(5, 3)
C	(2, 1)	(2, 0)	(6, 6)

Kalan sütunlarda **C**, **A**'yı kesin baskılar (u_1: 6 > 1) → A elenir.

Yinelemeli Eleme: 3×3 Örnek

Satır oyuncusu \ Sütun oyuncusu	X	Y	Z
A	(3, 2)	(3, 3)	(1, 4)
B	(3, 2)	(0, 4)	(5, 3)
C	(2, 1)	(2, 0)	**(6, 6)**

Tek hücre kaldı: **(C, Z) = (6, 6)**. Bu, oyunun tek Nash dengesidir (en iyi tepki işaretlemesi de aynı sonucu verir).

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3, 2)	(3, 3)	(1, 4)
B	(3, 2)	(0, 4)	(5, 3)
C	(2, 1)	(2, 0)	(6, 6)

Kalan satır ve sütun listeleri: **R = [0, 1, 2]** (A, B, C), **C = [0, 1, 2]** (X, Y, Z).

Adım 1 / 11

```
1  def ele(A, B):
▶      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
4          x = [r for r in R for s in R if
5               all(A[s][c] > A[r][c] for c in C)]
6          y = [c for c in C for d in C if
7               all(B[r][d] > B[r][c] for r in R)]
8          if x: R.remove(x[0])      # satır elenir
9          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

R = [0, 1, 2] C = [0, 1, 2]

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3 , 2)	(3 , 3)	(1 , 4)
B	(3 , 2)	(0 , 4)	(5 , 3)
C	(2 , 1)	(2 , 0)	(6 , 6)

Hiçbir satır baskılanmıyor (x = []). Sütun X, Z tarafından baskılanıyor → **y = [0]**.

Adım 2 / 11

```
1  def ele(A, B):
2      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
▶          x = [r for r in R for s in R if
▶               all(A[s][c] > A[r][c] for c in C)]
▶          y = [c for c in C for d in C if
▶               all(B[r][d] > B[r][c] for r in R)]
8          if x: R.remove(x[0])      # satır elenir
9          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

R = [0, 1, 2] C = [0, 1, 2]

x = [] y = [0]

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3 , 2)	(3 , 3)	(1 , 4)
B	(3 , 2)	(0 , 4)	(5 , 3)
C	(2 , 1)	(2 , 0)	(6 , 6)

C.remove(0): X sütunu silinir; sıradaki turda satırlar yeniden denetlenir.

Adım 3 / 11

```
1  def ele(A, B):
2      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
4          x = [r for r in R for s in R if
5               all(A[s][c] > A[r][c] for c in C)]
6          y = [c for c in C for d in C if
7               all(B[r][d] > B[r][c] for r in R)]
8          if x: R.remove(x[0])      # satır elenir
▶          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

`R = [0, 1, 2]` `C = [1, 2]` `x = []` `y = [0]`

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3 , 2)	(3 , 3)	(1 , 4)
B	(3 , 2)	(0 , 4)	(5 , 3)
C	(2 , 1)	(2 , 0)	(6 , 6)

Satır B, C tarafından baskılanıyor → **x = [1]**. Sütunlara da bakılır: y = [].

Adım 4 / 11

```
1  def ele(A, B):
2      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
▶          x = [r for r in R for s in R if
▶               all(A[s][c] > A[r][c] for c in C)]
▶          y = [c for c in C for d in C if
▶               all(B[r][d] > B[r][c] for r in R)]
8          if x: R.remove(x[0])      # satır elenir
9          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

R = [0, 1, 2] C = [1, 2]

x = [1] y = []

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3, 2)	(3, 3)	(1, 4)
B	(3, 2)	(0, 4)	(5, 3)
C	(2, 1)	(2, 0)	(6, 6)

R.remove(1): B satırı silinir, döngü küçülen oyunla yeniden başlar.

Adım 5 / 11

```
1  def ele(A, B):
2      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
4          x = [r for r in R for s in R if
5               all(A[s][c] > A[r][c] for c in C)]
6          y = [c for c in C for d in C if
7               all(B[r][d] > B[r][c] for r in R)]
▶          if x: R.remove(x[0])      # satır elenir
9          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

R = [0, 2] C = [1, 2] x = [1] y = []

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3 , 2)	(3 , 3)	(1 , 4)
B	(3 , 2)	(0 , 4)	(5 , 3)
C	(2 , 1)	(2 , 0)	(6 , 6)

Hiçbir satır baskılanmıyor (x = []). Sütun Y, Z tarafından baskılanıyor → **y = [1]**.

Adım 6 / 11

```
1  def ele(A, B):
2      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
▶          x = [r for r in R for s in R if
▶               all(A[s][c] > A[r][c] for c in C)]
▶          y = [c for c in C for d in C if
▶               all(B[r][d] > B[r][c] for r in R)]
8          if x: R.remove(x[0])      # satır elenir
9          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

R = [0, 2] C = [1, 2] x = [] y = [1]

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3 , 2)	(3 , 3)	(1 , 4)
B	(3 , 2)	(0 , 4)	(5 , 3)
C	(2 , 1)	(2 , 0)	(6 , 6)

C.remove(1): Y sütunu silinir; sıradaki turda satırlar yeniden denetlenir.

Adım 7 / 11

```
1  def ele(A, B):
2      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
4          x = [r for r in R for s in R if
5               all(A[s][c] > A[r][c] for c in C)]
6          y = [c for c in C for d in C if
7               all(B[r][d] > B[r][c] for r in R)]
8          if x: R.remove(x[0])      # satır elenir
▶          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

R = [0, 2] | C = [2] | x = [] | y = [1]

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3 , 2)	(3 , 3)	(1 , 4)
B	(3 , 2)	(0 , 4)	(5 , 3)
C	(2 , 1)	(2 , 0)	(6 , 6)

Satır A, C tarafından baskılanıyor → **x = [0]**. Sütunlara da bakılır: y = [].

Adım 8 / 11

```
1  def ele(A, B):
2      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
▶          x = [r for r in R for s in R if
▶               all(A[s][c] > A[r][c] for c in C)]
▶          y = [c for c in C for d in C if
▶               all(B[r][d] > B[r][c] for r in R)]
8          if x: R.remove(x[0])      # satır elenir
9          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

R = [0, 2] C = [2] x = [0] y = []

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3 , 2)	(3 , 3)	(1 , 4)
B	(3 , 2)	(0 , 4)	(5 , 3)
C	(2 , 1)	(2 , 0)	(6 , 6)

R.remove(0): A satırı silinir, döngü küçülen oyunla yeniden başlar.

Adım 9 / 11

```
1  def ele(A, B):
2      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
4          x = [r for r in R for s in R if
5               all(A[s][c] > A[r][c] for c in C)]
6          y = [c for c in C for d in C if
7               all(B[r][d] > B[r][c] for r in R)]
▶          if x: R.remove(x[0])      # satır elenir
9          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

R = [2] C = [2] x = [0] y = []

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3, 2)	(3, 3)	(1, 4)
B	(3, 2)	(0, 4)	(5, 3)
C	(2, 1)	(2, 0)	(6, 6)

Ne satır ne sütun baskılanıyor: **x = y = []**.

Adım 10 / 11

```
1  def ele(A, B):
2      R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3      while True:
▶          x = [r for r in R for s in R if
▶               all(A[s][c] > A[r][c] for c in C)]
▶          y = [c for c in C for d in C if
▶               all(B[r][d] > B[r][c] for r in R)]
8          if x: R.remove(x[0])      # satır elenir
9          elif y: C.remove(y[0])    # sütun elenir
10         else: return R, C
```

DEĞİŞKENLER

R = [2] | C = [2] | x = [] | y = []

Kod İzleme: Yinelemeli Baskın Strateji Eleme

	X	Y	Z
A	(3 , 2)	(3 , 3)	(1 , 4)
B	(3 , 2)	(0 , 4)	(5 , 3)
C	(2 , 1)	(2 , 0)	(6 , 6)

return [2], [2] → yalnızca (C, Z) kaldı. 4 elemede sırayla X, B, Y, A silindi.

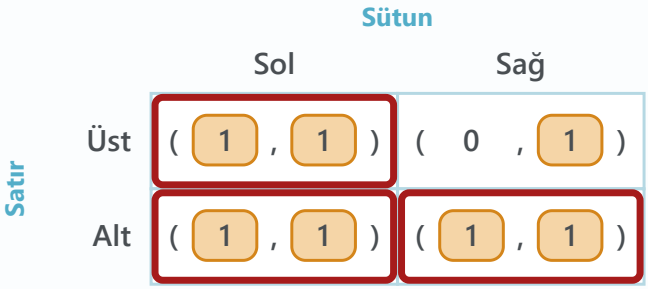
Adım 11 / 11

```
1 def ele(A, B):
2     R, C = [0, 1, 2], [0, 1, 2]   # kalanlar
3     while True:
4         x = [r for r in R for s in R if
5              all(A[s][c] > A[r][c] for c in C)]
6         y = [c for c in C for d in C if
7              all(B[r][d] > B[r][c] for r in R)]
8         if x: R.remove(x[0])      # satır elenir
9         elif y: C.remove(y[0])    # sütun elenir
▶         else: return R, C
```

DEĞİŞKENLER

R = [2] C = [2] x = [] y = []

Zayıf Baskılamanın Tuzağı



- Satır için Alt, Üst'ü **zayıf** baskılar (1 = 1, 1 > 0). Sütun oyuncusu her durumda kayıtsızdır.
- Saf Nash dengeleri: **(Üst, Sol), (Alt, Sol), (Alt, Sağ)** — üç tane.
- Üst silinirse (Üst, Sol) dengesi kaybolur; eleme sırası farklı sonuçlar bırakabilir.
- Ders: kesin baskılama güvenli; zayıf baskılama **denge kaybettirebilir**.

Kesin eleme her oyunda tek hücreye inmez: Cinsiyetler Savaşı'nda hiçbir strateji baskılanmaz. O zaman en iyi tepki işaretlemesine ve karışık stratejilere başvururuz.

KARIŞIK STRATEJİ

Karışık Strateji ve Kayıtsızlık İlkesi

- **Karışık strateji** σ_i: saf stratejiler üzerinde bir olasılık dağılımı. Kazanç = beklenen kazanç.
- Saf dengesi olmayan oyunlarda (Yazı-Tura, penaltı, denetim) oyuncular **tahmin edilemez** olmak ister.
- **Destek (support)**: pozitif olasılık verilen saf stratejiler.

Kayıtsızlık ilkesi: Dengede her oyuncu, desteğindeki tüm saf stratejilerden *aynı* beklenen kazancı alır (destek dışındakiler daha fazla vermez). Aksi hâlde daha iyi olana tüm olasılığı kaydırırdı.

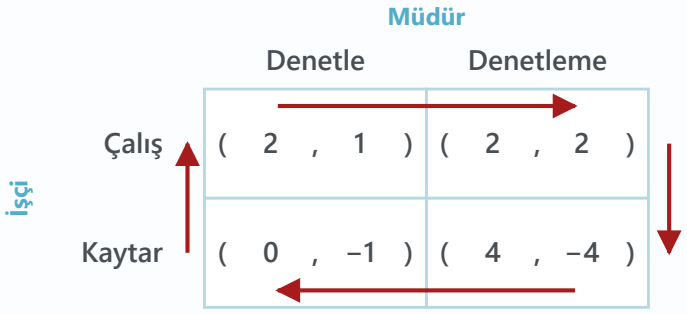
Şaşırtıcı sonuç

Satır oyuncusunun karışımı, *sütun oyuncusunun* kazançlarından hesaplanır (ve tersi).

Yorum

Olasılıklar "zar atmak" ya da büyük bir nüfustaki davranış oranları (ör. penaltıların yönü) olarak okunabilir.

Karışık Denge Hesabı: Denetim Oyunu



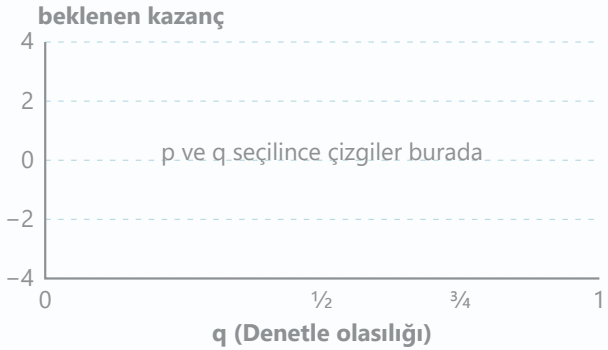
Denetim oyunu (inspection game)

Satır: işçi, sütun: müdür. Ücret 4, çaba maliyeti 2, üretim 6, denetim maliyeti 1. Kaytaran yakalanırsa ücret alamaz.

Oklar: her hücreden bir oyuncu kârlı sapıyor → en iyi tepkiler döngü oluşturur, **saf Nash dengesi yok**.

Karışık Denge Hesabı: Denetim Oyunu

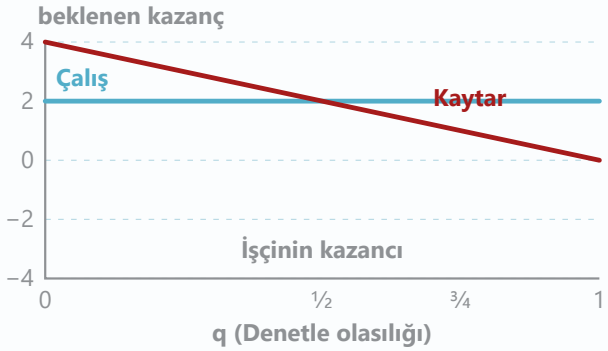
	Denetle	Denetleme
Çalış	(2 , 1)	(2 , 2)
Kaytar	(0 , −1)	(4 , −4)



1. İşçi **p** olasılıkla Çalış, müdür **q** olasılıkla Denetle oynar.

Karışık Denge Hesabı: Denetim Oyunu

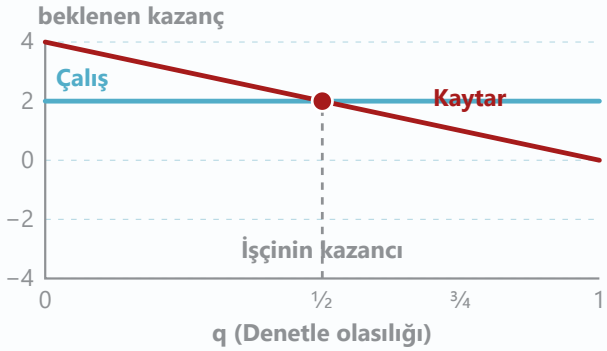
	Denetle	Denetleme
Çalış	(2 , 1)	(2 , 2)
Kaytar	(0 , −1)	(4 , −4)



1. İşçi **p** olasılıkla Çalış, müdür **q** olasılıkla Denetle oynar.
2. İşçi: E[Çalış] = 2q + 2(1−q) = **2**; E[Kaytar] = 0·q + 4(1−q) = **4 − 4q**

Karışık Denge Hesabı: Denetim Oyunu

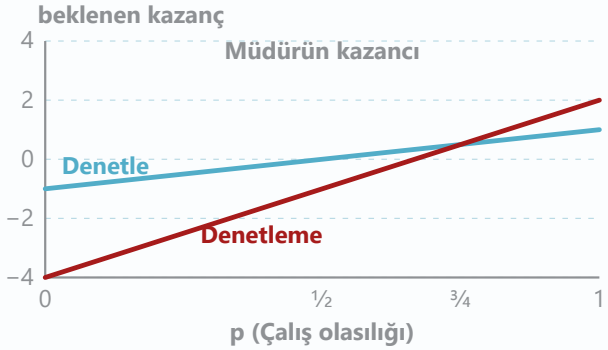
	Denetle	Denetleme
Çalış	(2 , 1)	(2 , 2)
Kaytar	(0 , −1)	(4 , −4)



1. İşçi **p** olasılıkla Çalış, müdür **q** olasılıkla Denetle oynar.
2. İşçi: E[Çalış] = 2q + 2(1−q) = **2**; E[Kaytar] = 0·q + 4(1−q) = **4 − 4q**
3. İşçi kayıtsız: 2 = 4 − 4q ⇒ **q* = 1/2**

Karışık Denge Hesabı: Denetim Oyunu

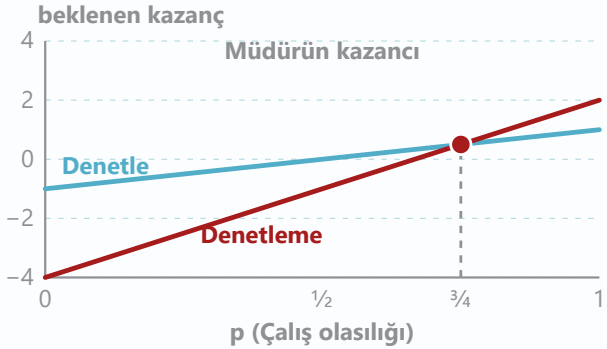
	Denetle	Denetleme
Çalış	(2 , 1)	(2 , 2)
Kaytar	(0 , −1)	(4 , −4)



1. İşçi **p** olasılıkla Çalış, müdür **q** olasılıkla Denetle oynar.
2. İşçi: E[Çalış] = 2q + 2(1−q) = **2**; E[Kaytar] = 0·q + 4(1−q) = **4 − 4q**
3. İşçi kayıtsız: 2 = 4 − 4q ⇒ **q* = 1/2**
4. Müdür: E[Denetle] = 1·p − 1·(1−p) = **2p − 1**; E[Denetleme] = 2p − 4(1−p) = **6p − 4**

Karışık Denge Hesabı: Denetim Oyunu

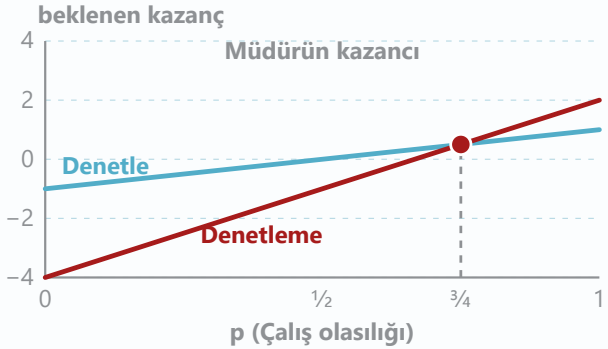
	Denetle	Denetleme
Çalış	(2 , 1)	(2 , 2)
Kaytar	(0 , −1)	(4 , −4)



1. İşçi **p** olasılıkla Çalış, müdür **q** olasılıkla Denetle oynar.
2. İşçi: E[Çalış] = 2q + 2(1−q) = **2**; E[Kaytar] = 0·q + 4(1−q) = **4 − 4q**
3. İşçi kayıtsız: 2 = 4 − 4q ⇒ **q* = 1/2**
4. Müdür: E[Denetle] = 1·p − 1·(1−p) = **2p − 1**; E[Denetleme] = 2p − 4(1−p) = **6p − 4**
5. Müdür kayıtsız: 2p − 1 = 6p − 4 ⇒ **p* = 3/4**

Karışık Denge Hesabı: Denetim Oyunu

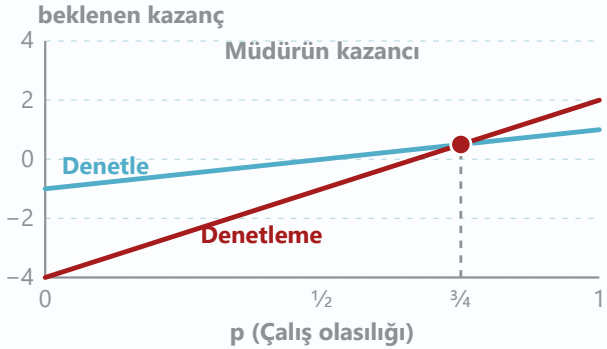
	Denetle	Denetleme
Çalış	(2 , 1)	(2 , 2)
Kaytar	(0 , −1)	(4 , −4)



1. İşçi **p** olasılıkla Çalış, müdür **q** olasılıkla Denetle oynar.
2. İşçi: E[Çalış] = 2q + 2(1−q) = **2**; E[Kaytar] = 0·q + 4(1−q) = **4 − 4q**
3. İşçi kayıtsız: 2 = 4 − 4q ⇒ **q* = 1/2**
4. Müdür: E[Denetle] = 1·p − 1·(1−p) = **2p − 1**; E[Denetleme] = 2p − 4(1−p) = **6p − 4**
5. Müdür kayıtsız: 2p − 1 = 6p − 4 ⇒ **p* = 3/4**
6. Denge: (p*, q*) = (3/4, 1/2). Beklenen kazanç: işçi 2, müdür 1/2.

Karışık Denge Hesabı: Denetim Oyunu

	Denetle	Denetleme
Çalış	(2 , 1)	(2 , 2)
Kaytar	(0 , −1)	(4 , −4)

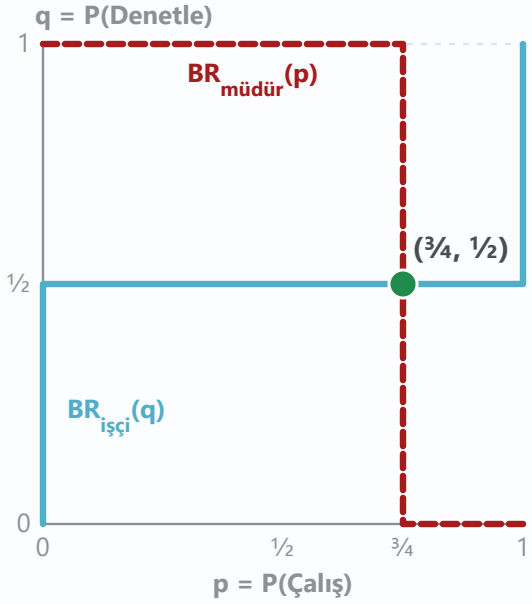


1. İşçi **p** olasılıkla Çalış, müdür **q** olasılıkla Denetle oynar.
2. İşçi: E[Çalış] = 2q + 2(1−q) = **2**; E[Kaytar] = 0·q + 4(1−q) = **4 − 4q**
3. İşçi kayıtsız: 2 = 4 − 4q ⇒ **q* = 1/2**
4. Müdür: E[Denetle] = 1·p − 1·(1−p) = **2p − 1**; E[Denetleme] = 2p − 4(1−p) = **6p − 4**
5. Müdür kayıtsız: 2p − 1 = 6p − 4 ⇒ **p* = 3/4**
6. Denge: (p*, q*) = (3/4, 1/2). Beklenen kazanç: işçi 2, müdür 1/2.
7. Ceza sertleşirse (yakalanan kaytarıcı −2): q* = 1/3 olur, p* = ¾ **değişmez**!

Nash'in Varlık Teoremi (1950)

Teorem. Sonlu sayıda oyuncu ve sonlu strateji kümesi olan **her** oyunun en az bir (karışık strateji) Nash dengesi vardır.

- İspat fikri: en iyi tepki eşlemesi $BR(\sigma) = (BR_1(\sigma_{-1}), \ldots)$ karışık profiller kümesinden kendisine gider; **Kakutani** (ya da Brouwer) sabit nokta teoremiyle $\sigma^* \in BR(\sigma^*)$ olan bir nokta vardır.
- Sabit nokta = herkesin herkese en iyi tepki verdiği profil = Nash dengesi.
- Garanti **karışık** stratejiler içindir; saf denge olmayabilir (Yazı-Tura).
- Sonsuz strateji kümelerinde denge olmayabilir (ör. "en büyük sayıyı söyleyen kazanır").



Denetim oyunu: iki en iyi tepki eğrisinin kesişimi = denge

Çoklu Denge: Geyik Avı

		Avcı 2: Geyik	Avcı 2: Tavşan
Avcı 1	Geyik	(4 , 4)	(0 , 3)
Avcı 1	Tavşan	(3 , 0)	(3 , 3)

Geyik Avı (Stag Hunt): geyik ancak birlikte avlanır (4); tavşan tek başına garanti 3 verir.

Oyuncu adları: satırda Avcı 1, sütunda Avcı 2; hücrede (Avcı 1, Avcı 2).

Çoklu Denge: Geyik Avı

Avcı 1 \ Avcı 2	Geyik	Tavşan
Geyik	(4, 4)	(0, 3)
Tavşan	(3, 0)	(3, 3)

En iyi tepkiler iki hücrede çakışır: **(Geyik, Geyik)** ve **(Tavşan, Tavşan)**. Hangisi oynanacak?

Oyuncu adları: satırda Avcı 1, sütunda Avcı 2; hücrede (Avcı 1, Avcı 2).

Çoklu Denge: Geyik Avı

Avcı 1 \ Avcı 2	Geyik	Tavşan
Geyik	(4, 4)	(0, 3)
Tavşan	(3, 0)	(3, 3)

Üçüncü denge karışık: karşı taraf Geyik'i q ile seçiyorsa 4q = 3 ⇒ **q = ¾**. Geyik ancak ortağın ≥ %75 olasılıkla geleceğine inanılırsa en iyi tepkidir.

Oyuncu adları: satırda Avcı 1, sütunda Avcı 2; hücrede (Avcı 1, Avcı 2).

Çoklu Denge: Geyik Avı

Avcı 1 \ Avcı 2	Geyik	Tavşan
Geyik	(4, 4)	(0, 3)
Tavşan	(3, 0)	(3, 3)

Pareto baskınlık: (Geyik, Geyik) iki avcıya da daha çok verir (4 > 3) → verimlilik ölçütü onu seçer.

Oyuncu adları: satırda Avcı 1, sütunda Avcı 2; hücrede (Avcı 1, Avcı 2).

Çoklu Denge: Geyik Avı

Avcı 1 \ Avcı 2	Geyik	Tavşan
Geyik	(4, 4)	(0, 3)
Tavşan	(3, 0)	(3, 3)

Risk baskınlık (Harsanyi–Selten): sapma kayıpları çarpımı (G,G): (4−3)·(4−3) = 1; (T,T): (3−0)·(3−0) = 9 → **(Tavşan, Tavşan)** daha "güvenli".

Oyuncu adları: satırda Avcı 1, sütunda Avcı 2; hücrede (Avcı 1, Avcı 2).

Denge Seçimi: Hangisi Oynanır?

Pareto baskınlık

Herkesi daha iyi yapan dengeyi seç. Geyik Avı'nda (G, G). Güven gerektirir.

Risk baskınlık

Karşı tarafın ne yapacağından emin değilsen kayıp riski düşük olanı seç. Geyik Avı'nda (T, T).

Odak noktası (Schelling, 1960)

Kültür, alışkanlık ya da belirginlik bir dengeyi öne çıkarır: "İstanbul'da buluşacaksak saat 12'de Taksim".

Korelasyonlu denge (correlated equilibrium)

Aumann (1974): ortak bir sinyal (trafik ışığı) oyuncuları dengelerden birine yönlendirir; kavşaktaki Tavuk oyununu çözer.

Öğrenme ve evrim

Tekrarlanan etkileşimde hangi dengeye varıldığı başlangıca bağlıdır (en iyi tepki dinamiği, replikatör).

Adalet sorunu

Cinsiyetler Savaşı'nda saf dengelerin biri bir oyuncuyu, diğeri öbürünü kayırır; karışık denge ise ikisine de yalnızca 6/5 verir.

Korelasyonlu Denge (Aumann, 1974): Trafik Işığı

Sürücü 1 \ Sürücü 2	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

Saf dengeler: (Kır, Düz), (Düz, Kır)

Karışık denge: kırma 9/10

çarpışma 1/100, kazanç −1/10

Tavuk (kavşak) oyunu: iki saf denge birini kayırır; simetrik karışık dengede %1 çarpışma olur ve beklenen kazanç −1/10'dur.

Korelasyonlu Denge (Aumann, 1974): Trafik Işığı

Sürücü 1 \ Sürücü 2	Kır	Düz git
Kır	(0, 0) μ = 1/3	(−1, 1) μ = 1/3
Düz git	(1, −1) μ = 1/3	(−10, −10) μ = 0

Işık 3 hücreden birini 1/3 ile seçer, her sürücüye YALNIZ kendi önerisini söyler.

Bir **aracı** (trafik ışığı) ortak olasılık dağılımı μ'den bir hücre çeker ve her sürücüye yalnız kendi önerisini fısıldar.

Korelasyonlu Denge (Aumann, 1974): Trafik Işığı

Sürücü 1 \ Sürücü 2	Kır	Düz git
Kır	(0, 0) μ = 1/3	(−1, 1) μ = 1/3
Düz git	(1, −1) μ = 1/3	(−10, −10) μ = 0

S1'e "Kır" dendi ⇒ S2: Kır ½, Düz ½

Uy (Kır): 0·½ + (−1)·½ = −½

Sap (Düz): 1·½ + (−10)·½ = −4,5

−½ > −4,5 ⇒ uymak iyi

Öneri "Kır" ise Bayes kuralıyla rakip ½–½ Kır/Düz; uymak (−½) sapmaktan (−4,5) iyidir.

Korelasyonlu Denge (Aumann, 1974): Trafik Işığı

Sürücü 1 \ Sürücü 2	Kır	Düz git
Kır	(0, 0) μ = 1/3	(−1, 1) μ = 1/3
Düz git	(1, −1) μ = 1/3	(−10, −10) μ = 0

S1'e "Düz git" dendi ⇒ S2 kesin Kır

Uy (Düz): 1 Sap (Kır): 0

1 > 0 ⇒ uymak iyi

Sürücü 2 için de simetrik.

Öneri "Düz git" ise rakibe kesin "Kır" denmiştir; uymak (1) sapmaktan (0) iyidir. Hiçbir öneriden sapmak kârlı değil ⇒ **korelasyonlu denge**.

Korelasyonlu Denge (Aumann, 1974): Trafik Işığı

Sürücü 1 \ Sürücü 2	Kır	Düz git
Kır	(0, 0) μ = 1/3	(−1, 1) μ = 1/3
Düz git	(1, −1) μ = 1/3	(−10, −10) μ = 0



Beklenen kazanç 0'ar (karışık dengedeki −1/10'dan iyi), çarpışma hiç yok. Her Nash dengesi bir korelasyonlu dengedir; tersi doğru değil. Korelasyonlu denge doğrusal programla polinom zamanda bulunur.

Sıfır Toplamlı Oyun: Maksimin ve Minimaks

Satır (maks.) \ Sütun (min.)	X	Y	Z
A	4	3	5
B	2	1	6
C	5	2	0

Sıfır toplamlı: hücrede yalnızca satır oyuncusunun kazancı var; sütun oyuncusu aynı miktarı **kaybeder** ($u_2 = -u_1$).

Sıfır Toplamlı Oyun: Maksimin ve Minimaks

Satır (maks.) \ Sütun (min.)	X	Y	Z	satır min.
A	4	**3**	5	3
B	2	**1**	6	1
C	5	2	**0**	0

Satır oyuncusu kötümser düşünür: her satırda rakibin ona bırakacağı **en kötü** değer: 3, 1, 0.

Sıfır Toplamlı Oyun: Maksimin ve Minimaks

Satır (maks.)	Sütun (min.) X	Y	Z	satır min.
A	4	3	5	3
B	2	1	6	1
C	5	2	0	0

Maksimin = max(3, 1, 0) = **3** (satır A). Satır oyuncusu en az 3'ü garanti eder.

Sıfır Toplamlı Oyun: Maksimin ve Minimaks

Satır (maks.) \ Sütun (min.)	X	Y	Z	satır min.
A	4	**3**	5	**3**
B	2	1	**6**	1
C	**5**	2	0	0
sütun maks.	5	3	6	

Sütun oyuncusu da kötümser: her sütunda satır oyuncusunun alabileceği **en yüksek** değer: 5, 3, 6.

Sıfır Toplamlı Oyun: Maksimin ve Minimaks

Satır (maks.) \ Sütun (min.)	X	Y	Z	satır min.
A	4	3	5	3
B	2	1	6	1
C	5	2	0	0
sütun maks.	5	3	6	

Minimaks = min(5, 3, 6) = **3** (sütun Y). Sütun oyuncusu en fazla 3 kaybetmeyi garanti eder.

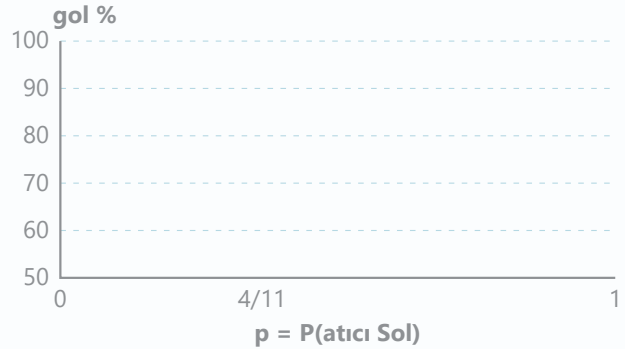
Sıfır Toplamlı Oyun: Maksimin ve Minimaks

Satır (maks.) \ Sütun (min.)	X	Y	Z	satır min.
A	4	**3**	5	**3**
B	2	1	6	1
C	5	2	0	0
sütun maks.	5	**3**	6	

Maksimin = minimaks = 3 ⇒ **eyer noktası** (A, Y): satırının en küçüğü, sütununun en büyüğü. Saf Nash dengesidir; **oyunun değeri v = 3**.

Eyer Noktası Yoksa: Penaltı Atışı

Atıcı \ Kaleci	Sol	Sağ	min
Sol	60	95	60
Sağ	90	70	70
maks	90	95	

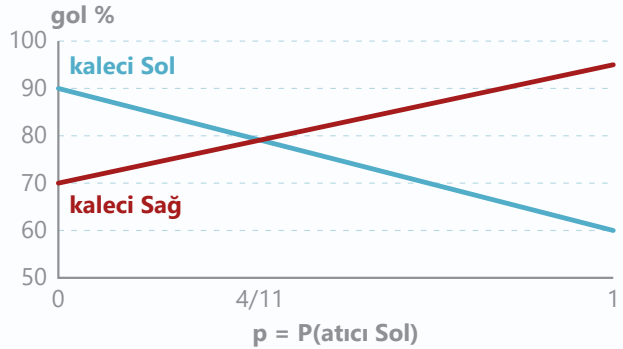


1. Satır min. 60, 70 → maksimin 70; sütun maks. 90, 95 → minimaks 90. 70 < 90: **eyer noktası yok**.

Örnek değerler (gerçek veride benzer yapı: Palacios-Huerta, 2003). Sol/Sağ atıcıya göredir.

Eyer Noktası Yoksa: Penaltı Atışı

Atıcı \ Kaleci	Sol	Sağ
Sol	60	95
Sağ	90	70

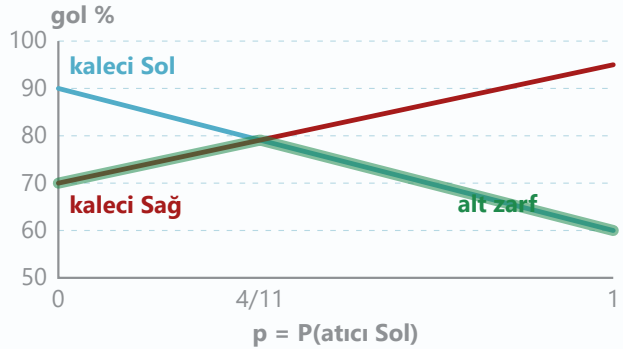


1. Satır min. 60, 70 → maksimin 70; sütun maks. 90, 95 → minimaks 90. 70 < 90: **eyer noktası yok**.
2. Atıcı p ile Sol: kaleci Sol ⇒ 60p + 90(1−p) = **90 − 30p**; kaleci Sağ ⇒ 95p + 70(1−p) = **70 + 25p**.

Örnek değerler (gerçek veride benzer yapı: Palacios-Huerta, 2003). Sol/Sağ atıcıya göredir.

Eyer Noktası Yoksa: Penaltı Atışı

Atıcı \ Kaleci	Sol	Sağ
Sol	60	95
Sağ	90	70

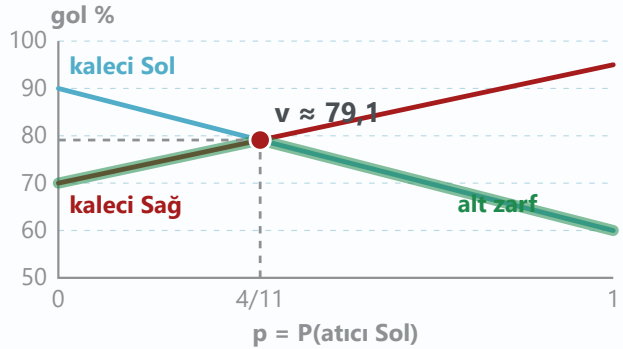


1. Satır min. 60, 70 → maksimin 70; sütun maks. 90, 95 → minimaks 90. 70 < 90: **eyer noktası yok**.
2. Atıcı p ile Sol: kaleci Sol ⇒ 60p + 90(1−p) = **90 − 30p**; kaleci Sağ ⇒ 95p + 70(1−p) = **70 + 25p**.
3. Kaleci gol yüzdesini en aza indirir: atıcının garantisi iki doğrunun **alt zarfı**.

Örnek değerler (gerçek veride benzer yapı: Palacios-Huerta, 2003). Sol/Sağ atıcıya göredir.

Eyer Noktası Yoksa: Penaltı Atışı

Atıcı \ Kaleci	Sol	Sağ
Sol	60	95
Sağ	90	70

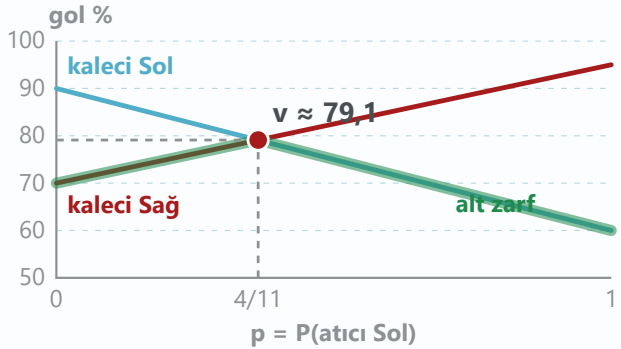


1. Satır min. 60, 70 → maksimin 70; sütun maks. 90, 95 → minimaks 90. 70 < 90: **eyer noktası yok**.
2. Atıcı p ile Sol: kaleci Sol ⇒ 60p + 90(1−p) = **90 − 30p**; kaleci Sağ ⇒ 95p + 70(1−p) = **70 + 25p**.
3. Kaleci gol yüzdesini en aza indirir: atıcının garantisi iki doğrunun **alt zarfı**.
4. Alt zarfın tepesi: 90 − 30p = 70 + 25p ⇒ **p* = 4/11 ≈ 0,36**, **v = 870/11 ≈ %79,1**.

Örnek değerler (gerçek veride benzer yapı: Palacios-Huerta, 2003). Sol/Sağ atıcıya göredir.

Eyer Noktası Yoksa: Penaltı Atışı

Atıcı \ Kaleci	Sol	Sağ
Sol	60	95
Sağ	90	70



1. Satır min. 60, 70 → maksimin 70; sütun maks. 90, 95 → minimaks 90. 70 < 90: **eyer noktası yok**.
2. Atıcı p ile Sol: kaleci Sol ⇒ 60p + 90(1−p) = **90 − 30p**; kaleci Sağ ⇒ 95p + 70(1−p) = **70 + 25p**.
3. Kaleci gol yüzdesini en aza indirir: atıcının garantisi iki doğrunun **alt zarfı**.
4. Alt zarfın tepesi: 90 − 30p = 70 + 25p ⇒ **p* = 4/11 ≈ 0,36**, **v = 870/11 ≈ %79,1**.
5. Kaleci q ile Sol: 60q + 95(1−q) = 90q + 70(1−q) ⇒ **q* = 5/11**; kalecinin garantisi de %79,1 ⇒ maks-min = min-maks.

Örnek değerler (gerçek veride benzer yapı: Palacios-Huerta, 2003). Sol/Sağ atıcıya göredir.

von Neumann Minimaks Teoremi (1928)

Her sonlu iki kişilik sıfır toplamlı oyunda $\max_x \min_y x^T A y = \min_y \max_x x^T A y = v$

Denge = minimaks stratejileri

(x*, y*) Nash dengesi ⇔ x* maksimin, y* minimaks stratejisi. Tüm dengelerde değer aynı: v.

Değiştirilebilirlik

İki denge (x, y) ve (x′, y′) ise (x, y′) de dengedir: koordinasyon sorunu yok.

Doğrusal programlama

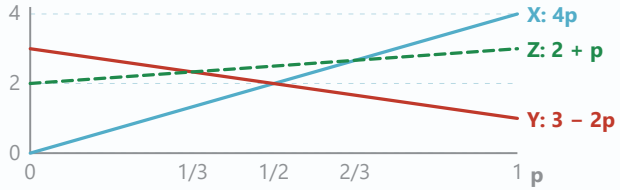
max v, öyle ki her j için $\Sigma_i x_i A_{ij} \geq v$, $\Sigma x_i = 1$, $x \geq 0$. LP polinom zamanda çözülür.

Öğrenme bağlantısı

İki oyuncu da pişmanlıksız algoritma (çarpımsal ağırlıklar) kullanırsa ortalama stratejiler minimaksa yakınsar.

Nash teoremi, minimaks teoreminin sıfır toplamlı olmayan n kişilik oyunlara genellemesi olarak görülebilir; ama genel oyunda değer tek değildir ve LP ile çözülemez.

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p

g(p)

A = [[4, 1, 3], [0, 3, 2]]: satır oyuncusu p ile 1. satırı oynarsa her sütun bir doğru verir. Saf maksimin 1, minimaks 3: eyer noktası yok.

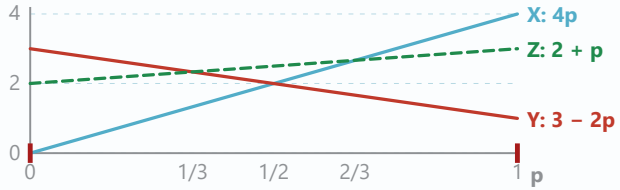
Adım 1 / 13

```
1  from fractions import Fraction as F
▶ def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
10     p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

DEĞİŞKENLER

—

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1
g(p)	·	·

n = 3 sütun. Adaylar önce uç noktalar: p = 0 ve p = 1.

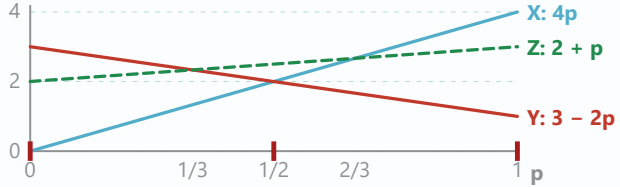
Adım 2 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
▶      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
10     p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

DEĞİŞKENLER

`n = 3` `aday = [0, 1]`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2
g(p)	·	·	·

Sütun X ile Y: d = 6 ≠ 0 ⇒ doğrular p = **1/2**'de kesişir, aday listesine eklenir.

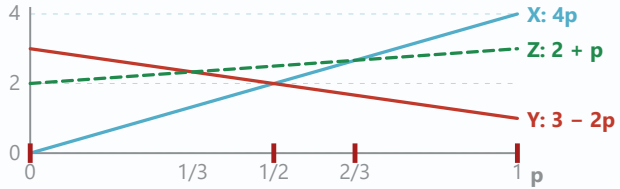
Adım 3 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
▶              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
▶              if d != 0:
▶                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
10     p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

DEĞİŞKENLER

`n = 3` `aday = [0, 1, 1/2]` `j = 0` `k = 1` `d = 6`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3
g(p)	·	·	·	·

Sütun X ile Z: d = 3 ≠ 0 ⇒ doğrular p = **2/3**'te kesişir, aday listesine eklenir.

Adım 4 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
▶              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
▶              if d != 0:
▶                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
10     p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

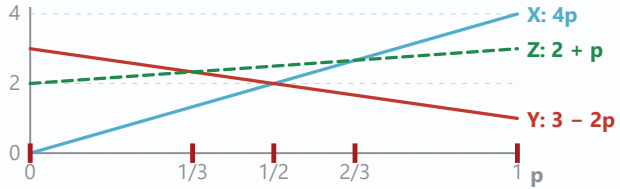
DEĞİŞKENLER

`n = 3`

`aday = [0, 1, 1/2, 2/3]`

`j = 0` `k = 2` `d = 3`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3	1/3
g(p)	·	·	·	·	·

Sütun Y ile Z: d = −3 ≠ 0 ⇒ doğrular p = **1/3**'te kesişir, aday listesine eklenir.

Adım 5 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
▶              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
▶              if d != 0:
▶                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
10     p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

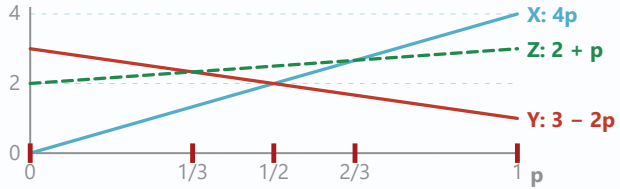
DEĞİŞKENLER

`n = 3`

`aday = [0, 1, 1/2, 2/3, 1/3]`

`j = 1` `k = 2` `d = -3`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3	1/3
g(p)	·	·	·	·	·

g(p) = $\min_j$ [p·A[0][j] + (1 − p)·A[1][j]]: sütun oyuncusu her p'ye en kötü sütunla yanıt verir (alt zarf).

Adım 6 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
▶      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
10     p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

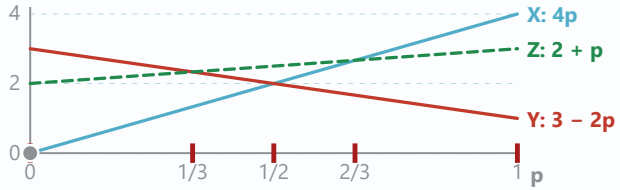
DEĞİŞKENLER

`n = 3`

`aday = [0, 1, 1/2, 2/3, 1/3]`

`j = 2` `k = 2` `d = -3`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3	1/3
g(p)	0	·	·	·	·

max, adayı dener: g(0) = 0. Şimdiye kadarki en iyi.

Adım 7 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
▶      p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

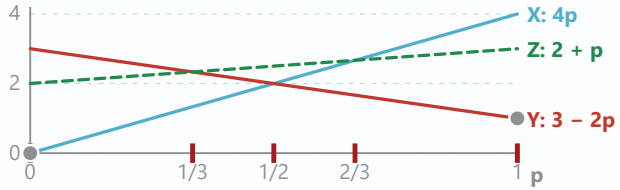
DEĞİŞKENLER

`n = 3`

`aday = [0, 1, 1/2, 2/3, 1/3]`

`j = 2` `k = 2` `d = -3`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3	1/3
g(p)	0	1	·	·	·

max, adayı dener: g(1) = 1. Şimdiye kadarki en iyi.

Adım 8 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
▶      p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

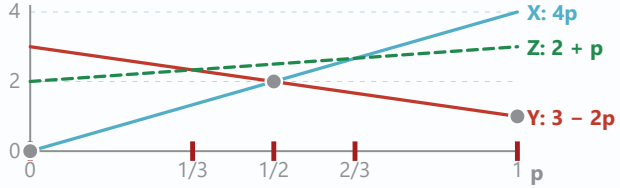
DEĞİŞKENLER

`n = 3`

`aday = [0, 1, 1/2, 2/3, 1/3]`

`j = 2` `k = 2` `d = -3`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3	1/3
g(p)	0	1	2	·	·

max, adayı dener: g(1/2) = 2. Şimdiye kadarki en iyi.

Adım 9 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
▶      p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

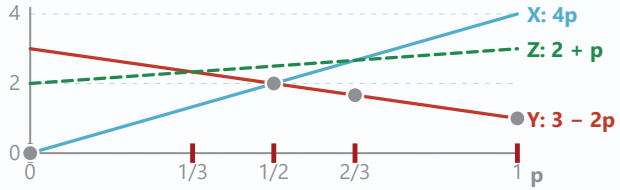
DEĞİŞKENLER

`n = 3`

`aday = [0, 1, 1/2, 2/3, 1/3]`

`j = 2` `k = 2` `d = -3`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3	1/3
g(p)	0	1	2	5/3	·

max, adayı dener: g(2/3) = 5/3.

Adım 10 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
▶      p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

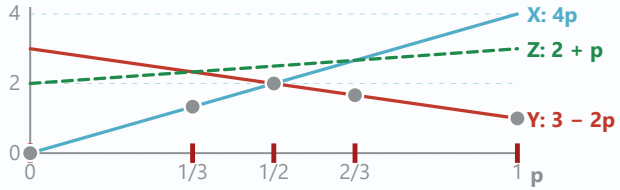
DEĞİŞKENLER

`n = 3`

`aday = [0, 1, 1/2, 2/3, 1/3]`

`j = 2` `k = 2` `d = -3`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3	1/3
g(p)	0	1	2	5/3	4/3

max, adayı dener: g(1/3) = 4/3.

Adım 11 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
▶      p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

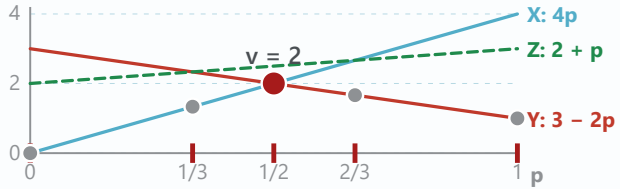
DEĞİŞKENLER

`n = 3`

`aday = [0, 1, 1/2, 2/3, 1/3]`

`j = 2` `k = 2` `d = -3`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3	1/3
g(p)	0	1	2	5/3	4/3

En büyük garanti p = 1/2'de: **v = 2**. Z sütununun doğrusu zarfın üstünde kalır: sütun oyuncusu Z'yi hiç oynamaz.

Adım 12 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
▶      p = max((c for c in aday if 0 <= c <= 1), key=g)
11     return p, g(p)
```

DEĞİŞKENLER

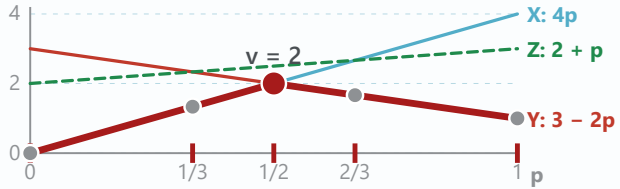
`n = 3`

`aday = [0, 1, 1/2, 2/3, 1/3]`

`j = 2` `k = 2` `d = −3`

`p = 1/2`

Kod İzleme: `maksimin(A)` — 2 × n Sıfır Toplamlı



aday p	0	1	1/2	2/3	1/3
g(p)	0	1	2	5/3	4/3

Dönüş (1/2, 2). Minimaks tarafı: sütun oyuncusu X'i 1/3, Y'yi 2/3 ile oynarsa satır en fazla 2 alır ⇒ max-min = min-max = 2.

Adım 13 / 13

```
1  from fractions import Fraction as F
2  def maksimin(A):                    # A: 2 x n
3      n, aday = len(A[0]), [F(0), F(1)]
4      for j in range(n):
5          for k in range(j + 1, n):
6              d = A[0][j] - A[1][j] - A[0][k] + A[1][k]
7              if d != 0:
8                  aday.append(F(A[1][k] - A[1][j], d))
9      g = lambda p: min(p*a+(1-p)*b for a, b in zip(*A))
10     p = max((c for c in aday if 0 <= c <= 1), key=g)
▶      return p, g(p)
```

DEĞİŞKENLER

`n = 3`

`aday = [0, 1, 1/2, 2/3, 1/3]`

`j = 2` `k = 2` `d = −3`

`p = 1/2` `dönüş = (1/2, 2)`

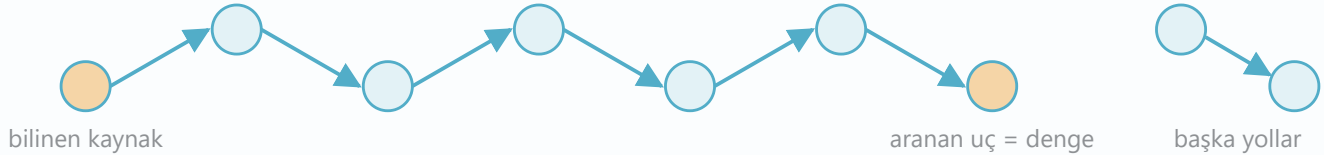
Dengeyi Hesaplamak Ne Kadar Zor?

Problem	Karmaşıklık	Yöntem / kaynak
Saf Nash dengesi (matris verili)	Polinom	En iyi tepki işaretleme, O(mn)
İki kişilik sıfır toplamlı, karışık	Polinom	Doğrusal programlama (minimaks)
İki kişilik genel oyun, karışık	**PPAD-tam**	Chen–Deng (2006); Lemke–Howson en kötü durumda üstel
Üç ve daha çok oyuncu	**PPAD-tam** (yaklaşık denge)	Daskalakis–Goldberg–Papadimitriou (2006)
En yüksek refahlı denge / tek denge mi?	NP-zor	Gilboa–Zemel (1989); Conitzer–Sandholm (2008)

- **Destek sayımı** (support enumeration): her destek çifti için kayıtsızlık denklemlerini çöz → destek sayısı kadar üstel.
- Üç oyunculu oyunlarda tek denge **irrasyonel** olasılıklar içerebilir; bu yüzden ε-yaklaşık denge aranır.

PPAD: Var Olduğu Bilinen Çözümü Bulmak

- Nash dengesi **her zaman vardır** → "denge var mı?" sorusu önemsiz; NP-tamlık bu tür (çözümü garanti) problemler için uygun değildir.
- **PPAD** (Papadimitriou, 1994): varlığı bir *parite argümanıyla* kanıtlanan problemler. Tam problem: **END-OF-LINE**.



- Her düğümün en fazla bir giren, bir çıkan oku var; dev çizge devre ile örtük verilir. Bir kaynaktan yürüyünce bir uca varılır — ama yol üstel uzunlukta olabilir.
- Sperner lemması, Brouwer sabit noktası ve Nash dengesi bu sınıfa girer. Sonuç: genel oyunlarda denge verimli bulunamayabilir → **oyuncular dengeyi "bulur" mu?**

Pratikte: Yaklaşık Denge ve Hesaplama Araçları

- **ε-Nash dengesi**: kazançlar [0, 1] aralığına ölçeklenmişken hiçbir oyuncu tek taraflı sapmayla ε'dan fazla kazanamaz. Gerçek zamanlı sistemler (reklam açık artırmaları, piyasalar) çoğu zaman buna yetinir.
- **Basit ½-yaklaşık denge**: satır oyuncusu bir i satırı seçer; sütun oyuncusu i'ye en iyi tepki j'yi; satır oyuncusu j'ye en iyi tepki k'yi bulur. (½ i + ½ k, j) bir ½-Nash dengesidir (Daskalakis–Mehta–Papadimitriou, 2006).
- **Küçük destekler yeter**: her ε için $O(\log n / \varepsilon^2)$ destekli bir ε-denge vardır → $n^{O(\log n / \varepsilon^2)}$ zamanda bulunur (Lipton–Markakis–Mehta, 2003).

```python
import nashpy as nash
A = [[2, 2], [0, 4]]    # işçi: Çalış / Kaytar
B = [[1, 2], [-1, -4]]  # müdür: Denetle / Denetleme
oyun = nash.Game(A, B)
for x, y in oyun.support_enumeration():
    print(x, y)  # [0.75 0.25] [0.5 0.5]
```

Denetim oyunu Nashpy ile: destek sayımı p* = 3/4, q* = 1/2 dengesini verir. Gambit de aynı işi (destek sayımı, Lemke–Howson, LP) arayüzle yapar.

Nash dengesinin ötesinde: **korelasyonlu denge** (LP ile polinom), **evrimsel kararlı strateji** (Maynard Smith–Price, 1973), sıralı oyunlarda **alt oyun yetkin denge** (Selten, 1965) ve öğrenmeyle ulaşılan **kaba korelasyonlu denge**.

ÖZET

Tartışma: Denge, Adalet ve Rasyonellik

Denge adil mi?

Hayır, gerekmez. Denge yalnızca karşılıklı en iyi tepkidir: Cinsiyetler Savaşı'nda kazançlar eşit değildir, Mahkûmun İkilemi'nde herkes için daha iyi bir sonuç vardır. *Adalet ≠ denge*.

İnsanlar rasyonel mi?

Gerçek kararlar bilgi eksikliği, zaman baskısı, duygular ve sosyal normlardan etkilenir (sınırlı rasyonellik, Herbert Simon). Model, stratejik bağımlılığı incelemek için açık bir karşılaştırma noktasıdır.

Etik sınırlar

Aldatma, manipülasyon, adil olmayan avantaj… "Kazanmak için her yol mubah mı?" Stratejik davranışın sınırlarını kurallar ve kurumlar çizer — bu da mekanizma tasarımının konusudur.

ÖZET

Özet

Kavram	Ne söyler?	Nasıl bulunur?
Nash dengesi	Kimse tek başına sapmak istemez	Karşılıklı en iyi tepki
Baskın strateji eleme	Rasyonel oyuncu baskılananı oynamaz	Kesin baskılananları yinelemeli sil
Karışık denge	Destekteki stratejiler eşit kazandırır	Kayıtsızlık denklemleri
Varlık (Nash 1950)	Sonlu oyunda karışık denge hep var	Kakutani / Brouwer sabit noktası
Çoklu denge	Kuram tek sonucu seçmeyebilir	Pareto / risk baskınlık, odak noktası
Korelasyonlu denge	Aracının önerisine uymak herkesin çıkarına	Uyma kısıtları + doğrusal programlama
Minimaks	Sıfır toplamlıda maks-min = min-maks = v	Eyer noktası ya da LP
Karmaşıklık	Genel oyunda denge PPAD-tam	Lemke–Howson, destek sayımı

Örnek Problemler

Oyun 1	L	M	R
U	(2, 1)	(1, 2)	(0, 0)
D	(3, 0)	(0, 1)	(1, 3)

Oyun 2	L	R
U	(4, 1)	(0, 3)
D	(1, 2)	(2, 0)

1. Oyun 1'in saf Nash dengelerini en iyi tepki işaretlemesiyle bulun.
2. Oyun 2'nin karışık dengesini kayıtsızlık ilkesiyle bulun.
3. Sıfır toplamlı A = [[3, −1, 2], [1, 0, 4], [−2, −3, 5]]: eyer noktası var mı? Değer?
4. “Her Nash dengesi Pareto optimaldir.” Doğru mu?
5. Varlık teoremi neden karışık stratejiler için ifade edilir?
6. Cournot, P = 15 − Q, maliyet 0: denge? Kartel?

▶ **Cevapları göster**

ÖZET

Etkileşimli Oyunlar ve Laboratuvarlar

Matris ve Nash Laboratuvarı

Herhangi bir matriste en iyi tepkiler ve dengeler.

Nashpy ve Gambit

Dengeyi bilgisayarla bulma: destek sayımı, Lemke–Howson.

Albay Blotto

Birlik dağıtma: saf dengesi olmayan sıfır toplamlı oyun.

Mahkûmun İkilemi

Baskın strateji ve denge simülatörü.

Taş-Kağıt-Makas

Karışık denge: 1/3–1/3–1/3.

Satranç

Mükemmel bilgili sıfır toplamlı oyun ve yapay zekâ.

Tilki ve Kazlar

Asimetrik strateji oyunu.

Dokuz Taş

Klasik strateji oyunu.

Kaynaklar

- J. F. Nash, "Equilibrium points in n-person games", *PNAS* 36(1), 1950; "Non-cooperative games", *Annals of Mathematics* 54(2), 1951.
- J. von Neumann, "Zur Theorie der Gesellschaftsspiele", *Mathematische Annalen* 100, 1928.
- M. J. Osborne, *An Introduction to Game Theory*, Oxford University Press, 2004 (Bölüm 2–4).
- N. Nisan, T. Roughgarden, É. Tardos, V. Vazirani (ed.), *Algorithmic Game Theory*, Cambridge, 2007 (Bölüm 1–3).
- T. Roughgarden, *Twenty Lectures on Algorithmic Game Theory*, Cambridge, 2016 (Ders 18–20).
- C. Daskalakis, P. Goldberg, C. Papadimitriou, "The complexity of computing a Nash equilibrium", *SIAM J. Computing* 39(1), 2009.
- J. C. Harsanyi, R. Selten, *A General Theory of Equilibrium Selection in Games*, MIT Press, 1988.
- A. R. Karlin, Y. Peres, *Game Theory, Alive*, AMS, 2017 (sıfır toplamlı ve genel oyunlar, Nash teoremi).
- D. Easley, J. Kleinberg, *Networks, Crowds, and Markets*, Cambridge, 2010 (Bölüm 6: Oyunlar).
- Ders sayfaları: Roughgarden CS364A · Shah CSC304 · Araçlar: Gambit, Nashpy · Video: Game Theory 101

Son

Bölüm 3: Nash Dengesi

Sonraki bölüm: Bölüm 4: Mekanizma Tasarımı

Çalışma soruları · Terim sözlüğü